Employee Dataset

Data Cleaning & Preprocessing

Standardisasi & Validasi Data Karyawan untuk Akurasi Analisis Lanjutan

Presentation by

Joko Santoso

Agust 2026

Contact : santosoj160@gmail.com

Overview & Scope Analysis

Highlight Materi

1. Handling Duplikat & Standardisasi
2. Outlier Handling (IQR Method)
3. Missing Value Imputation
4. Feature Encoding

Dataset

Total Data : 1.248 Baris

Fitur : 14 Kolom

Cakupan :

Profil Karyawan, Metrik Kinerja, & Status Kerja

Sumber Data & Notebook

Dataset :

Employee Performance Data

Full Python Notebook:

Data_Cleaning_Joko.ipynb

Exploratory Data Analysis & Handling Duplikat

Handling Duplikat:

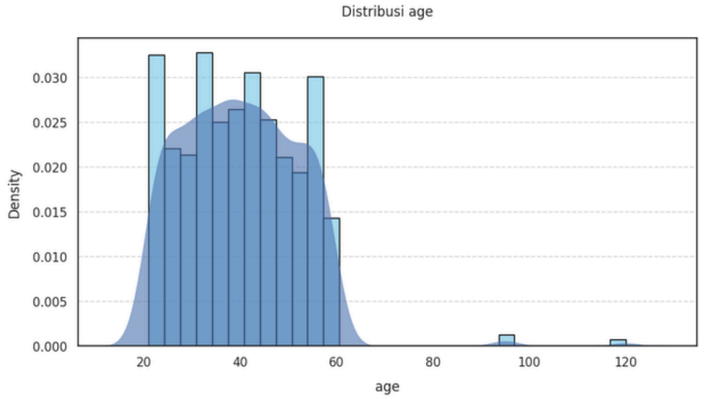
```
# jumlah duplikat
print("Jumlah duplicate:", df.duplicated().sum())

# hapus duplicate
df = df.drop_duplicates()

print("Setelah drop duplicate:", df.duplicated().sum())
```

```
Jumlah duplicate: 43
Setelah drop duplicate: 0
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1205 entries, 0 to 1237
Data columns (total 14 columns):
```

Terdeteksi anomali Usia



Standardisasi Teks

```
df["department"] = df["department"].replace("it", "IT")

df["department"].value_counts()
```

department	count
HR	221
Operations	218
IT	201
Sales	195
Marketing	185

```
df["city"] = df["city"].str.strip().str.lower()
df["city"] = df["city"].replace("jakarta", "Jakarta")

df["city"].value_counts()
```

city	count
surabaya	216
Jakarta	216
semarang	198

Kondisi awal dataset:

1248 Baris dan 14 Kolom

Terdeteksi anomali Usia

90 s.d 120 tahun

Handling Duplikat:

Hapus 43 baris ganda

Standardisasi Teks

"it" jadi "IT"

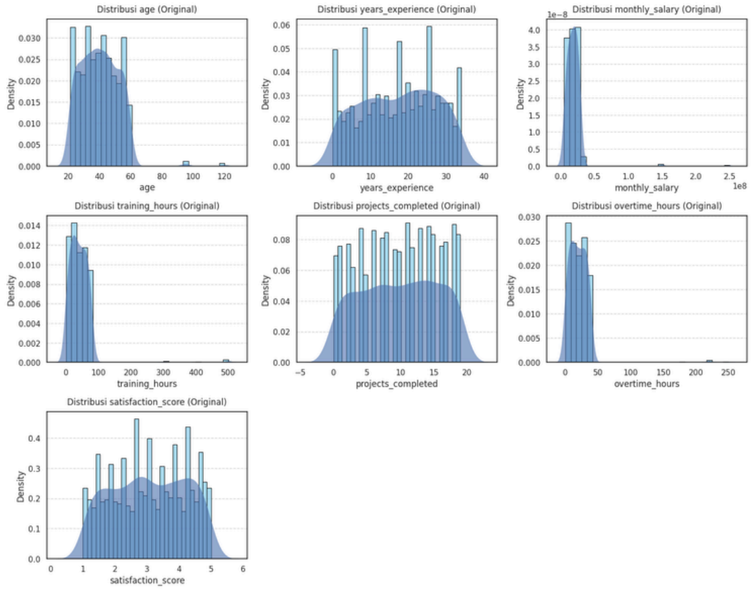
"jakarta" jadi "Jakarta"

Lihat Detail (Data_Cleaning_Joko.ipynb)

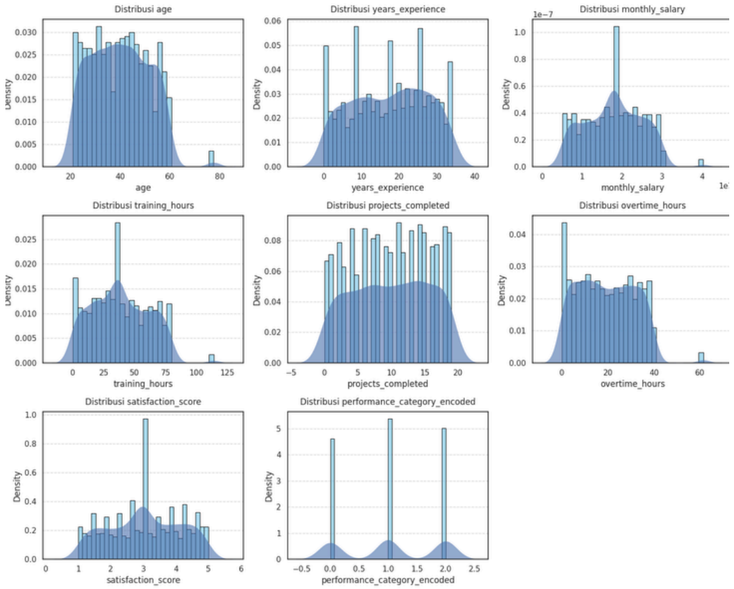
Outlier Analysis & Handling

	Kolom	Q1	Q3	IQR	Batas Bawah	Batas Atas	Jumlah Outlier
0	age	30.000000	49.000000	19.000000	1.500000	77.500000	8
1	years_experience	9.000000	26.000000	17.000000	-16.500000	51.500000	0
2	monthly_salary	12210423.000000	23434261.000000	11223838.000000	-4625334.000000	40270018.000000	8
3	training_hours	19.000000	57.000000	38.000000	-38.000000	114.000000	8
4	projects_completed	5.000000	15.000000	10.000000	-10.000000	30.000000	0
5	overtime_hours	9.000000	30.000000	21.000000	-22.500000	61.500000	8
6	satisfaction_score	2.200000	3.900000	1.700000	-0.350000	6.450000	0

Before



After



Capping / Winsorization Method

Usia (age):

Mencapai 120 tahun (Batas normal Q3 + (1.5 x IQR). = 77.5 tahun).

Jam Lembur (overtime_hours):

Nilai maks hingga 250 jam / bulan.

Jam Pelatihan (training_hours):

Nilai maks hingga 500 jam.

Gaji (monthly_salary)

Terdeteksi nilai ekstrem di atas batas IQR.

🔗 Lihat Detail (Data_Cleaning_Joko.ipynb)

Missing Value Analysis & Handling

Before

```
# ngecek jumlah missing value per kolom
missing_count = df.isnull().sum()

# hitung persen missing value
missing_percent = (df.isnull().sum() / len(df)) * 100

# gabungkan hasil ke tabel
missing_df = pd.DataFrame({
    "Missing Count": missing_count,
    "Missing Percent (%)": missing_percent
})

# nmpilkan hanya kolom yang ada missing value
missing_df[missing_df["Missing Count"] > 0].sort_values(by="Missing
    Percent (%)", ascending=False)
```

	Missing Count	Missing Percent (%)
education_level	98	8.132780
monthly_salary	97	8.049793
satisfaction_score	97	8.049793
training_hours	96	7.966805

After

```
# mastikne gak ada missing value tersisa
df.isnull().sum()
```

	0
employee_id	0
department	0
job_role	0
education_level	0
work_mode	0
city	0
age	0
years_experience	0
monthly_salary	0
training_hours	0
projects_completed	0
overtime_hours	0
satisfaction_score	0
performance_category	0

dtype: int64

Missing Value

Terdeteksi ~8% data kosong

Fitur Numerik → Median

Fitur Kategorikal → Modus

Lihat detail (Missing Value **& Handling**)

Lihat Detail (Data_Cleaning_Joko.ipynb)

Feature Encoding

Label Encoding

```
# Melakukan Label Encoding pada performance_category
df["performance_category_encoded"] = le.fit_transform
    (df["performance_category"])

# Menampilkan mapping hasil encoding
print("\n=== MAPPING LABEL ENCODING ===")
for i, class_ in enumerate(le.classes_):
    print(f"{class_} -> {i}")
```

```
=== MAPPING LABEL ENCODING ===
High -> 0
Low -> 1
Medium -> 2
```

```
# Import Label Encoder
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

# Menampilkan data sebelum encoding
print("=== SEBELUM ENCODING ===")
print(df["performance_category"].head())
```

```
=== SEBELUM ENCODING ===
0    Low
1    Low
2    Low
3    Low
4    Low
Name: performance_category, dtype: object
```

```
# Menampilkan hasil setelah encoding
print("\n=== SESUDAH ENCODING ===")
df[["performance_category", "performance_category_encoded"]].head()
```

```
=== SESUDAH ENCODING ===
```

	performance_category	performance_category_encoded
0	Low	1
1	Low	1
2	Low	1
3	Low	1
4	Low	1

One-Hot Encoding

```
=== SEBELUM FEATURE ENCODING (Categorical Text Data) ===
```

Tabel Data Kategorikal Teks

	department	work_mode	performance_category
0	IT	Hybrid	Low
1	IT	Hybrid	Low
2	Sales	Remote	Low
3	IT	Remote	Low
4	IT	Onsite	Low

```
============================================================

=== SESUDAH FEATURE ENCODING (Full Numerical Data) ===
```

Tabel Hasil Label & One-Hot Encoding

	performance_category_encoded	department_Sales	work_mode_Onsite	work_mode_Remote
0	1	False	False	False
1	1	False	False	False
2	1	True	False	True
3	1	False	False	True
4	1	False	True	False

Data nominal → Kolom biner (drop_first=True).

department & work_mode

Fitur naik dari 15 jadi 20 kolom (100% numerik).

🔗 Lihat Detail (Data_Cleaning_Joko.ipynb)

Thank You!

Any thoughts or reflections?

Agust 2026

Contact : santosoj160@gmail.com